

# Manual Completo: Conectar la Vista de Usuarios al Backend (Spring Boot)

---

Este manual está hecho para tu proyecto actual y explica todo desde cero, paso por paso, con mucho detalle.

Archivo de la vista a conectar:

- `src/components/users/Users.jsx`

Objetivo:

- Reemplazar los datos mock por datos reales del backend.
- Conectar botones de acción (editar, cambiar rol, desactivar).
- Manejar carga, errores y mensajes visuales.

---

## 1. Verificar que el proyecto frontend funciona antes de tocar código

1. Abre una terminal en la raíz del proyecto.
2. Ejecuta instalación de dependencias (si aún no lo hiciste):

```
npm install
```

3. Levanta el frontend:

```
npm run dev
```

4. Confirma que abre en navegador (normalmente `http://localhost:5173`).
5. Entra a la pantalla de usuarios y verifica que la UI carga correctamente.

Si esto no funciona, arregla primero frontend local antes de conectar backend.

---

## 2. Verificar que el backend Spring Boot está levantado

1. Levanta tu backend.
2. Confirma el puerto (ejemplo común: 8080).
3. Prueba endpoint de usuarios desde navegador o Postman:

```
http://localhost:8080/api/users
```

4. Si responde JSON, puedes continuar.
5. Si no responde, revisa backend antes de seguir.

---

### 3. Crear variable de entorno en Vite para la URL del backend

Vite requiere prefijo VITE\_ para variables que usa React.

1. En la raíz del proyecto crea el archivo .env (si no existe).
2. Agrega:

```
VITE_API_URL=http://localhost:8080
```

3. Guarda el archivo.
4. Reinicia npm run dev (muy importante, Vite no toma cambios de .env sin reiniciar).

---

### 4. Modificar Users.jsx para consumir API

Archivo a editar:

- src/components/users/Users.jsx

#### 4.1 Cambiar import de React

Reemplaza:

```
import React, { useState } from 'react'
```

Por:

```
import React, { useEffect, useState } from 'react'
```

#### 4.2 Mantener mock opcionalmente, pero ya no renderizarlo

Puedes dejar mockUsers como respaldo temporal, pero el render debe usar estado users.

#### 4.3 Agregar estados nuevos y URL base

Dentro del componente Users, debajo de searchTerm, agrega:

```
const [users, setUsers] = useState([])
const [loading, setLoading] = useState(false)
const [error, setError] = useState('')

const API_URL = import.meta.env.VITE_API_URL || 'http://localhost:8080'
```

## 4.4 Agregar función para cargar usuarios (GET)

Dentro del componente Users, agrega:

```
const loadUsers = async () => {
  try {
    setLoading(true)
    setError('')

    const response = await fetch(`${API_URL}/api/users`)
    if (!response.ok) {
      throw new Error(`Error al cargar usuarios. HTTP ${response.status}`)
    }

    const data = await response.json()

    // Normaliza para que la tabla use siempre los mismos campos
    const normalizedUsers = data.map((u) => ({
      id: u.id,
      name: u.name ?? `${u.firstName ?? ''} ${u.lastName ?? ''}`.trim(),
      email: u.email ?? 'sin-correo',
      role: u.role ?? 'Sin rol',
      status: u.status ?? (u.active ? 'Activo' : 'Inactivo'),
      avatar: u.avatar || `https://i.pravatar.cc/150?u=${u.id}`
    })))

    setUsers(normalizedUsers)
  } catch (err) {
    setError(err.message || 'No se pudieron cargar los usuarios')
  } finally {
    setLoading(false)
  }
}
```

## 4.5 Ejecutar carga automática al entrar a la pantalla

Agrega este bloque dentro del componente Users:

```
useEffect(() => {
  loadUsers()
}, [])
```

---

## 5. Reemplazar la tabla para usar users en vez de mockUsers

En el render de la tabla cambia:

1. Donde hoy haces map/filter sobre mockUsers, usa users.
2. En el contador de abajo cambia mockUsers.length por users.length.

Ejemplo:

```
{users
  .filter(
    (user) =>
      user.name.toLowerCase().includes(searchTerm.toLowerCase()) ||
      user.email.toLowerCase().includes(searchTerm.toLowerCase())
  )
  .map((user) => (
    // fila
  )))}
```

Footer:

```
Mostrando {users.length} usuarios
```

---

## 6. Mostrar estado de carga, error y lista vacía

Antes de la tabla agrega mensajes:

```
{error && <div className="alert alert-danger mb-3">{error}</div>}
{loading && <div className="small text-secondary mb-2">Cargando usuarios...
</div>}
{!loading && !error && users.length === 0 && (
  <div className="small text-secondary mb-2">No hay usuarios para mostrar.
</div>
)}
```

Esto evita que la UI quede en blanco sin explicación.

---

## 7. Conectar botón Desactivar usuario (PATCH)

### 7.1 Función

Agrega dentro del componente Users:

```
const handleDeactivate = async (userId) => {
  try {
    setError('')

    const response = await fetch(`${API_URL}/api/users/${userId}/status`, {
      method: 'PATCH',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ status: 'Inactivo' })
    })
  } catch (error) {
    setError(error.message)
  }
}
```

```
    })

    if (!response.ok) {
      throw new Error(`No se pudo desactivar. HTTP ${response.status}`)
    }

    await loadUsers()
  } catch (err) {
    setError(err.message || 'Error al desactivar usuario')
  }
}
```

## 7.2 Enlazar al botón rojo

En el botón desactivar agrega:

```
onClick={() => handleDeactivate(user.id)}
```

---

## 8. Conectar botón Cambiar rol (PATCH)

### 8.1 Función

```
const handleChangeRole = async (userId, newRole) => {
  try {
    setError('')

    const response = await fetch(`${API_URL}/api/users/${userId}/role`, {
      method: 'PATCH',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ role: newRole })
    })

    if (!response.ok) {
      throw new Error(`No se pudo cambiar rol. HTTP ${response.status}`)
    }

    await loadUsers()
  } catch (err) {
    setError(err.message || 'Error al cambiar rol')
  }
}
```

### 8.2 Enlazar al botón amarillo

```
onClick={() => handleChangeRole(user.id, 'Supervisor')}
```

Nota:

- Ese valor es de prueba.
- Lo correcto después es abrir un modal o select con roles válidos.

---

## 9. Conectar botón Editar perfil (PUT)

### 9.1 Función

```
const handleEditUser = async (userId, payload) => {
  try {
    setError('')

    const response = await fetch(`${API_URL}/api/users/${userId}`, {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(payload)
    })

    if (!response.ok) {
      throw new Error(`No se pudo editar usuario. HTTP ${response.status}`)
    }

    await loadUsers()
  } catch (err) {
    setError(err.message || 'Error al editar usuario')
  }
}
```

### 9.2 Enlazar al botón azul

```
onClick={() =>
  handleEditUser(user.id, {
    name: user.name,
    email: user.email,
    role: user.role
  })
}
```

Nota:

- Esta versión reenvía valores actuales.
- Lo ideal es abrir formulario/modal y enviar cambios reales.

---

## 10. Ajustar CORS en Spring Boot si el navegador bloquea peticiones

Síntoma:

- En consola del navegador aparece error CORS.

Solución general:

1. Permitir origen del frontend en backend (ejemplo `http://localhost:5173`).
2. Aplicarlo globalmente o en el controlador de usuarios.

Si no habilitas CORS, frontend no podrá hablar con backend aunque la API exista.

---

## 11. Validación completa después de integrar

Checklist final:

1. Abres vista de usuarios y carga información real del backend.
  2. Buscador filtra correctamente sobre users.
  3. Botón desactivar cambia estado y refresca tabla.
  4. Botón cambiar rol actualiza y refresca.
  5. Botón editar actualiza y refresca.
  6. Si backend está caído, se muestra mensaje de error.
  7. Si backend tarda, aparece Cargando usuarios.
  8. Si no hay usuarios, aparece mensaje de lista vacía.
- 

## 12. Errores comunes y cómo resolverlos

### 1. Error 404

- Ruta incorrecta.
- Revisa URL exacta de endpoints y prefijo `/api`.

### 2. Error 401 o 403

- API protegida.
- Debes enviar token Authorization.

### 3. Error 415 Unsupported Media Type

- Falta header Content-Type: `application/json`.

### 4. Error 500 backend

- Problema del servidor.
- Revisar logs de Spring Boot.

### 5. Tabla vacía sin error

- Mapeo de campos incorrecto.

- Ajusta name, email, role, status según JSON real.
- 

## 13. Recomendación para dejarlo profesional

1. Crear carpeta src/services.
  2. Mover llamadas fetch a un archivo apiUsers.js.
  3. Dejar Users.jsx solo para UI y estado.
  4. Agregar confirmación antes de desactivar.
  5. Agregar modal real para editar/cambiar rol.
- 

## 14. Plantilla mínima funcional consolidada (resumen)

Si quieres hacerlo rápido, estos son los elementos obligatorios:

1. useEffect para llamar loadUsers.
2. Estado users, loading y error.
3. fetch GET para listar.
4. fetch PATCH/PUT para acciones.
5. Reemplazar mockUsers por users en render.
6. Variable VITE\_API\_URL configurada.

Con eso ya queda conectada la vista al backend.